



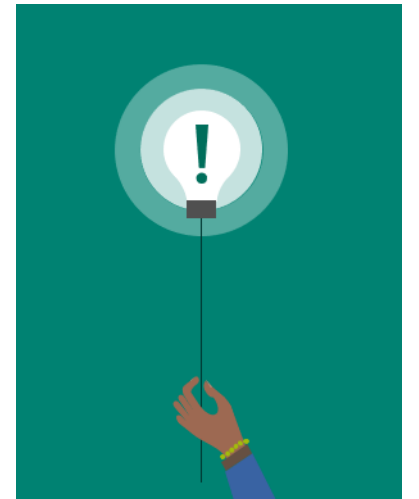
Classes

Module 4

Objectives

After completing this learning, you will be able to:

- Understand what a Class is
- Work With Methods
- Work with Advanced Object Features



Classes Overview

What are classes

- Custom data types can be created with sets of unique properties and methods
 - Requires PowerShell v5+
- Create instances of unique class just like any other data type
 - Supports static members as well

What do they help with

- Packaging up output in a more **professional** way than normal [`PSCustomObject`] data
- Making it **easier** and **faster** to create multiple copies of custom objects
- Enabling creation of a suite of tools by **standardizing output/input**
 - Ex. *-Service all support returning or receiving [`ServiceController`] objects
- Adding custom methods to data will avoid extra functions in scripts
- Differentiate between objects with the same properties

Simplification and Standardization

Using a PSCustomObject

```
$person = [pscustomobject]{  
    Name= "Gadi"  
    age = 50  
    Office= "Home"  
    Title= "CSA"  
    Planet= "Earth"  
    Dimension= "c137"  
}
```

```
$person.GetType().fullname
```

```
System.Management.Automation.PSCustomObject
```

Annoying / Unclear

- Pscustomobject returned
- Pscustomobject full name
- Hard to reuse / duplicate

Simplification and Standardization (Continued)

Using a PowerShell Class

```
$person = [person]::new("Gadi", 50, "home", "CSA", "Earth", "C137")
```

```
$person | GM
```

```
TypeName: person
```

Name	MemberType	Definition
----	-----	-----
...		
age	Property	int age {get;set;}
dimension	Property	string dimension {get;set;}
Name	Property	string Name {get;set;}
Office	Property	string Office {get;set;}
planet	Property	string planet {get;set;}
title	Property	string title {get;set;}

Concise/Precise

- Objects Class returned
- Objects Class full name
- easy to reuse / duplicate

Demonstration

Classes Introduction



Questions?



Class Basics

Class Syntax

Create a class

- Use the "Class" keyword
- Any custom name you want
- Open and closed curly braces

Declaration of type

Name of the class

```
class Person  
{  
  
}
```

Code brackets to contain the class

Class Syntax cont.

Add Properties (information)

Strongly type data if you want

- Default is [Object]

Variable will be the property name

```
class Person
{
    # Public Properties
    [String] $Name
    [Int32] $Age
}
```

Class Syntax cont. (Continued)

Add Properties (information)

Strongly type data if you want

- Default is `[Object]`

Variable will be the property name

Add Methods (actions)

Strongly type return data if you have any

- Default is `[void]`

Friendly name with Parenthesis

Must use "Return" to pass out data

```
class Person
{
    # Public Properties
    [String] $Name
    [Int32] $Age

    #public methods
    [String] SaySomething()
    {
        return "Something!"
    }
}
```

Creating an Instance of your Class

New-Object

```
$person = New-Object -TypeName Person  
$person.name = "Kory"  
$person.age = 28
```

New-Object using a hash table

```
$vals = @{  
    name = "Kory"  
    age = 28  
}  
$person = New-Object -TypeName Person -Property $vals
```

Default Constructor

```
$person = [person]::new()  
$person.name = "Kory"  
$person.age = 28
```

Syntax covered later
in this module.

Demonstration

Class Basics



Questions?



Doing More With Methods

Class and their methods

- Methods can be declared within a class
 - Method can then be called just like a normal object
- A method makes it possible to execute a piece of code from the class
- Classes can declare multiple methods to **overload** them

Methods with Parameters

Param() is not allowed in a class method

- Listed inside the parenthesis next to the name

```
[String] SaySomething($something)
{
    return "$Something!"
}
```

- Supports strong typing

```
[String] SaySomething([string]$something)
```

- Used like any other method

```
$person.SaySomething("That was so funny")
>That was so funny!
```

Overloading a Method

Need multiple input variations for the same method?

Define multiple methods with the same name, but different parameters

Think of it like parameter sets for methods

```
[String] SaySomething($something)
{
    return "$Something!"
}

[String] SaySomething()
{
    return "Something!"
}

> $person.SaySomething()
> Something!

> $person.SaySomething("That was so funny")
> That was so funny!
```

\$This



Variable used in classes to interact with its' own members



Variable that contains the whole object



One major use: prevent duplicate code in method overloads



Used to leverage current data in methods, or change properties from a method

\$This – example with Methods

SaySomething(\$something)

- Will return its input

SaySomething()

- Just returns "Something!"

```
Class Person
{
    [String] SaySomething($something)
    {
        return "$Something!"
    }

    [String] SaySomething()
    {
        return "Something!"
    }
}

$person = New-Object -TypeName person
$person.SaySomething()
>Something!
```

\$This – example with Methods cont.

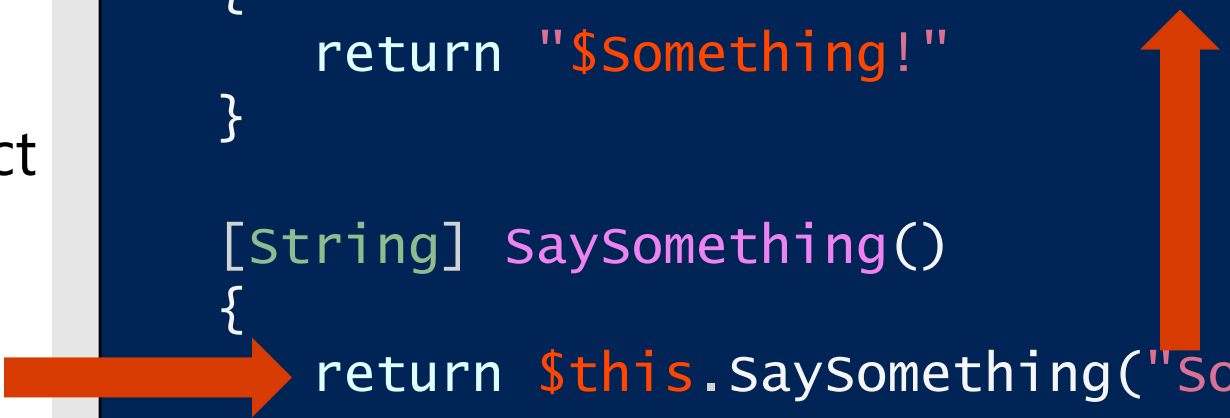
No need to initialize the object or repeat code

Using \$this and calling the method in need

```
Class Person
{
    [String] SaySomething($something)
    {
        return "$Something!"
    }

    [String] SaySomething()
    {
        return $this.SaySomething("Something")
    }
}

$person = New-Object -TypeName person
$person.SaySomething()
>Something!
```



\$This - Example with Properties

- **\$This** used in "GrowOlder" **Method** to update the "Age" **property**

```
class Person
{
    [int]$Age

    [void] GrowOlder([int32] $years)
    {
        $this.Age += $years
    }
}

> $person.Age = 28
> $person.GrowOlder(10)
> $person.Age
> 38
```

Demonstration

Creating Class Methods



Questions?



Advanced Object Features

Classes – Advanced features

Classes can use:

- Constructors
- Static members
- Hidden members
- Enums

Classes can inherit members from other classes

Best practice: when using many classes declare them in a separate script file

Basic Constructors

- Method called to create your object
- Default one takes in **no** params and sets **no** values

```
[person]::new()  
$person.name = "Kory"  
$person.age = 28
```

- Can redefine to set default values
 - Name must match class name
 - No return value specified – it knows you are giving back your custom class
 - Use \$this to touch the properties

```
person()  
{  
    $this.Name = "Kory"  
    $this.Age = 28  
}  
  
$person = [person]::new()  
$person.name  
> Kory
```

Constructor Overloads

- Can take in **parameter values** (usually to define properties on creation)
- **Overload** them like methods
- Use **\$this** to touch the properties

```
person ([String] $Name, [int32] $Age)
{
    $this.Name = $Name
    $this.Age = $Age
}
```

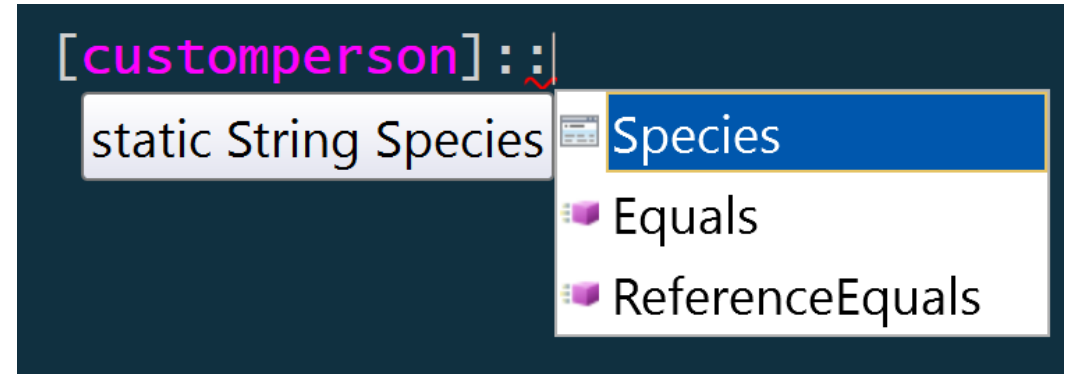
```
$person = [person]::new("Kory",28)
$person
```

```
> Name   Age
> ----   ---
> Kory   28
```

Static members

- Live on the type itself
 - Properties that never change
 - Methods that don't need \$This

Instances need to use \$Data::Property, not \$Data.Property



```
class person
{
...
  # Static Properties
  static [string] $Species = "Totally real human"
...
}

[person]::Species
> Totally real human
```

Hidden Members

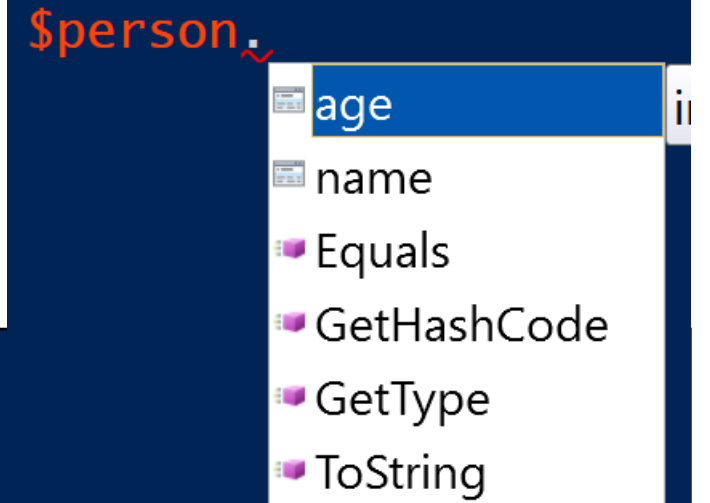
Hidden from get-member and auto complete

Accessible like any other member

-Force flag will show them on get-member

```
class person
{
...
  # Hidden Properties
  hidden [String] $RealSpecies = "Cephalopod"
...
}
```

```
$person.RealSpecies
> Cephalopod
```



Hidden Members (Continued)

Default hidden PS members

\$person GM	
Name	MemberType

Equals	Method
GetHashCode	Method
GetType	Method
SaySomething	Method
ToString	Method
Age	Property
Name	Property



Auto
Generated
hidden
methods



Our hidden
Property

\$person GM -Force	
Name	MemberType

pstypenames	CodeProperty
psadapted	MemberSet
psbase	MemberSet
psextended	MemberSet
psobject	MemberSet
Equals	Method
GetHashCode	Method
GetType	Method
SaySomething	Method
get_Age	Method
get_Name	Method
get_RealSpecies	Method
set_Age	Method
set_Name	Method
set_RealSpecies	Method
ToString	Method
Age	Property
Name	Property
RealSpecies	Property

Inheritance

- Create a modified version of an existing class
- Gets **most** existing members, which you can override

Only default Constructor is
inherited

- Able to add new members

Inheritance - Example

```
class person
{
  ...
  person()
  {
    $this.name = "Temp Name"
    $this.age = 30
  }
  ...
}

$person = [person]::new()
$person

> name      age
> ----      ---
```

```
class Cronenberg : person
{
  # Static Members
  static [String]$Species = "Human"
  # Hidden Properties
  hidden [String]$RealSpecies =
"Cronenberg"
}

$person = [Cronenberg]::new()
$person

> name      age
> ----      ---
> Temp Name  30
```

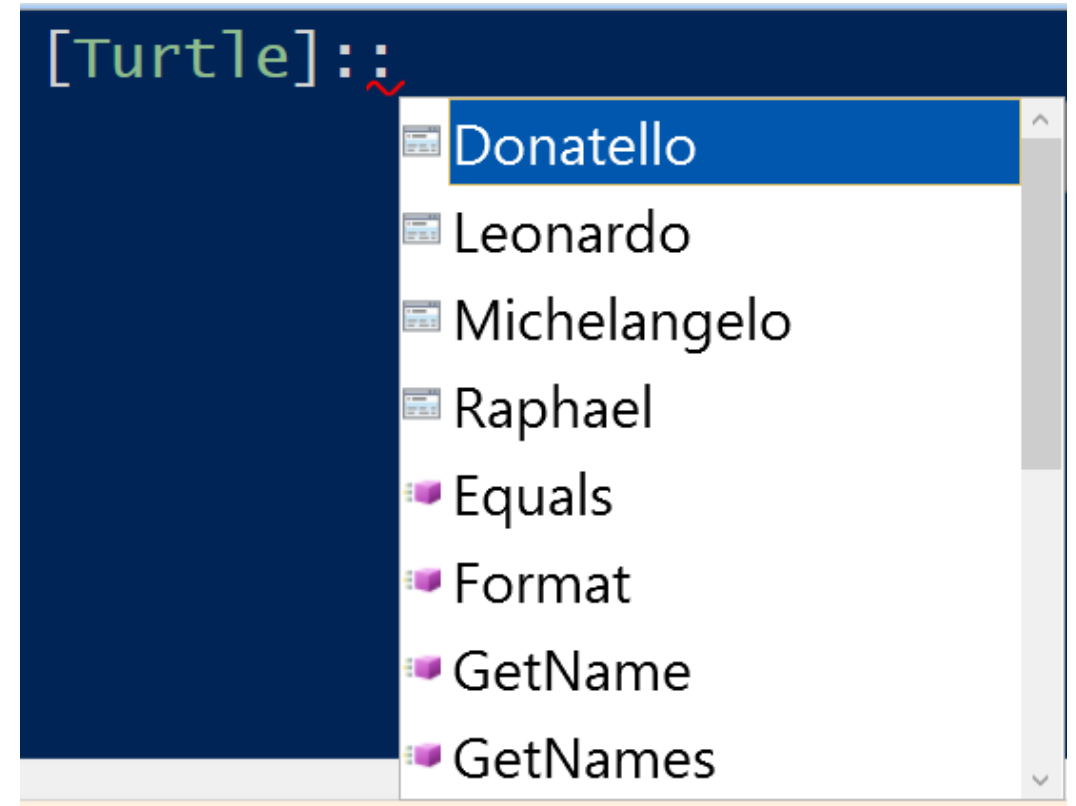
Can call base class constructors

Subclass (arguments) : base (argumentsToBaseClassConstructor) { ... }

Enums

- Basic sub types for pre-defined values
- Can be created within a class or separately

```
Enum Turtle
{
    Donatello
    Leonardo
    Michelangelo
    Raphael
}
```



Enums (Continued)

```
Enum Turtle
{
    Donatello
    Leonardo
    Michelangelo
    Raphael
}
```

```
function TurtlesInTest
{
    param([turtle]$Turtle)
    Switch($Turtle)
    {
        ...
    }
}
```

```
TurtlesInTest -Turtle 'Shredder'
```

Cannot process argument transformation on parameter 'Turtle'. Cannot convert value "Shredder" to type "Turtle". Error: "Unable to match the identifier name Shredder to a valid enumerator name. Specify one of the following enumerator names and try again: Donatello, Leonardo, Michelangelo, Raphael"

Load into Scripts

To reuse custom class, can keep it in its own file and "Dot-Source" it from scripts

```
. $PSScriptRoot\MyPerson.PS1  
  
$person = [person]::new("Kory",29)
```

Demonstration

Advanced Features



Questions?



PowerShell Classes

